

<https://gemini.google.com/share/760fed5094a7>

<https://gemini.google.com/share/9e8dfcb67f81>

## PROGRAM-1

**Title:** Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup.

### Viva / Theory Questions

**Q1. What is the main purpose of Build Automation Tools like Maven and Gradle?** **Ans:** Build automation tools automate the process of compiling source code, packaging binaries (like JAR/WAR files), managing external dependencies (libraries), and running automated tests. This ensures consistency and eliminates manual compilation errors.

**Q2. Mention two key differences between Maven and Gradle.** **Ans:** 1.

**Configuration Style:** Maven uses rigid, structured XML (pom.xml), whereas Gradle uses shorter, script-based files written in Groovy or Kotlin DSL (build.gradle).

2. **Performance:** Gradle is significantly faster than Maven because it supports incremental builds (only builds what has changed) and utilizes a build cache.

---

### Step-by-Step Procedure

#### Phase 1: Installing Eclipse IDE

- **Step 1:** Download the Eclipse installer from the official website: <https://www.eclipse.org/downloads/>.
- **Step 2:** Run the installer and select either **Eclipse IDE for Java Developers** or **Eclipse IDE for Enterprise Java and Web Developers**.
- **Step 3:** Select your desired installation folder and ensure the correct Java version path is detected. Click **Install**.
- **Step 4:** Once the installation finishes, launch Eclipse and select your workspace directory.

#### Phase 2: Installing and Setting Up Apache Maven

- **Step 5: Prerequisite (JDK Installation):** Ensure the current JDK version is installed on your Windows system.
  - Set the JAVA\_HOME environment variable pointing to your JDK installation folder.
  - Add the JDK bin path to your system's PATH environment variable.
- **Step 6: Download Maven:** Go to <https://maven.apache.org/download.cgi> and download the Binary zip archive (apache-maven-3.9.9-bin.zip).
- **Step 7: Extract Files:** Extract the downloaded ZIP archive directly into C:\Program Files (or use Command Prompt: unzip apache-maven-3.9.9-bin.zip).
- **Step 8: Environment Variables:** 1. Open Environment Variables settings in Windows.

2. Under System Variables, select **Path** and click **Edit**.

3. Add the path to the Maven bin directory (e.g., C:\Program Files\apache-maven-3.9.9\bin).

- **Step 9: Verification:** Open a new Command Prompt (cmd) and type:

DOS

```
mvn --version
```

*(Verify that the Maven version, Java version, and OS details are displayed properly).*

### Phase 3: Installing and Setting Up Gradle

- **Step 10: Create Folder:** Open File Explorer and create a new directory at the root level named C:\Gradle.
- **Step 11: Extract Distribution:** Extract the downloaded Gradle distribution folder (e.g., gradle-8.12.1) into your newly created C:\Gradle folder so the path looks like C:\Gradle\gradle-8.12.1.
- **Step 12: Environment Variables:** 1. Open Environment Variables settings.  
2. Under System Variables, edit the **Path** variable.  
3. Add the path to the Gradle bin directory (e.g., C:\Gradle\gradle-8.12.1\bin).
- **Step 13: Verification:** Open Command Prompt and type:

DOS

```
gradle -v
```

*(Verify that the Gradle version and environment details are printed on the screen).*

---

That happens because Microsoft Word doesn't always recognize the standard markdown arrow symbol (\$\rightarrow\$) correctly when pasting text directly from a browser.

To fix this for your document, I will use a simple, universal text arrow (->) from now on. Here is Program 2 rewritten with Word-friendly text arrows so you can copy and paste it cleanly:

---

## PROGRAM-2

**Title:** Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins.

### Viva / Theory Questions

**Q1. What is a POM file in Maven and what does it contain? Ans:** POM stands for **Project Object Model**. It is an XML file (pom.xml) located in the project's root directory. It contains configuration information for the project, including external dependencies (libraries), plugins, project version, and build configurations.

**Q2. What are 'Group Id' and 'Artifact Id' in a Maven project? Ans:** \* **Group Id:** Uniquely identifies your project across all projects (usually follows Java package naming conventions, e.g., com.company.project).

- **Artifact Id:** The name of the final jar/war file generated by the project (the name of your specific project module).
- 

### Step-by-Step Procedure

#### Phase 1: Creating a Maven Project in Eclipse

- **Step 1:** Open Eclipse IDE and navigate to:

**File -> New -> Other... -> Expand Maven -> Select Maven Project** and click **Next**.

- **Step 2:** In the wizard screen, ensure the checkbox **"Use default Workspace Location"** is checked. Click **Next**.
- **Step 3:** In the Archetype selection screen:
  - Set the Catalog to **Internal** (or type apache in the **Filter** box).
  - Scroll down and select **maven-archetype-quickstart** (this is used to create a simple, standard Java JAR application). Click **Next**.
- **Step 4:** Enter the project metadata details exactly as required:
  - **Group Id:** com.program2.maven
  - **Artifact Id:** program2-example-jar
  - **Version:** Leave as default (0.0.1-SNAPSHOT)
  - **Package:** com.program2.maven.program2
- **Step 5:** Click **Finish**.
- **Step 6:** Watch the Eclipse console for the automated build process. If the console asks for confirmation configuration, type **Y** and press **Enter** to confirm the properties (groupId, artifactId, etc.).

## Phase 2: Building and Packaging the Project

- **Step 7:** Expand your project directory in the Project Explorer to locate the automatically generated pom.xml file.
- **Step 8:** Right-click on the project folder -> Navigate to **Run As -> Select Maven build...**
- **Step 9:** In the configuration window that pops up, find the **Goals** text box, type **package**, and click **Run**.
- **Step 10:** Monitor the Eclipse console. Wait until you see the **"BUILD SUCCESS"** message, which indicates that Maven has compiled the code and bundled it into a JAR file.

## Phase 3: Running the Java Application

- **Step 11:** Expand the project directories: **src/main/java -> Open the package com.program2.maven.program2**.
- **Step 12:** Double-click and open the generated **App.java** file.
- **Step 13:** Right-click inside the App.java code editor -> Select **Run As -> Java Application**.
- **Step 14:** Verify the output (typically prints *"Hello World!"*) in the console window.

## PROGRAM-3

**Title:** Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation.

### Viva / Theory Questions

**Q1. What is the purpose of the gradle init command? Ans:** The gradle init command is an interactive CLI tool used to initialize and bootstrap a brand-new Gradle project. It guides the user through creating the necessary folder structures, build configuration files (build.gradle), and wrapper scripts based on the project type selected.

**Q2. What is the Gradle Wrapper (gradlew), and why is it preferred over a local Gradle installation? Ans:** The Gradle Wrapper (gradlew for Linux/macOS or gradlew.bat for Windows) is a script that automatically downloads and uses a specific declared version of Gradle for that project. This ensures build consistency

across different developer machines and CI/CD pipelines without requiring manual installation of Gradle on every system.

---

## Step-by-Step Procedure

### Phase 1: Initializing the Gradle Project via Command Line

- **Step 1:** Open the Windows Command Prompt (cmd).
- **Step 2:** Create a new folder for the program and navigate into it using the following commands:

DOS

```
mkdir pgm3
```

```
cd pgm3
```

- **Step 3:** Start the interactive Gradle setup by executing:

DOS

```
gradle init
```

- **Step 4:** Respond to the interactive prompts sequentially as specified by the manual:
  - **Select build type:** Select options for **Application** (Choose option 1).
  - **Select implementation language:** Select **Groovy**.
  - **Enter Java version:** Provide your installed JDK version when prompted.
  - **Enter Project name:** Enter your project name (e.g., pgm3).
  - **Select application structure:** Select **Single application project**.
  - **Select build script DSL:** Select **Kotlin** as the Domain Specific Language.
- **Step 5:** Once the configuration finishes, Gradle will generate the project structure and display a "**BUILD SUCCESSFUL**" message.

### Phase 2: Building and Running the Application

- **Step 6:** Compile and run the automatically generated application code by using the Gradle Wrapper script:

DOS

```
gradlew run
```

- **Step 7:** Wait for 3-5 minutes during the initial execution as Gradle downloads dependencies and configures the build script for the first time.
- **Step 8:** Verify that the application executes successfully and prints its output in the command window.

### Phase 3: Viewing Project Structure (Optional)

- **Step 9:** To see the complete tree structure of the generated files and directories, run the following command in the prompt:

DOS

```
tree /f
```

---

## PROGRAM-4

**Title:** Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle.

**Viva / Theory Questions**

### Q1. How does Gradle handle migration from an existing Maven project? Ans:

Gradle features native support for automated migrations. When gradle init is executed inside a directory containing a Maven pom.xml file, Gradle automatically detects it, parses the structure, and converts the dependencies, plugins, and configurations into corresponding Gradle build files.

### Q2. What is the benefit of running gradle clean build instead of just gradle build? Ans:

The clean task deletes the build directory entirely, removing all previously compiled classes and cached artifacts. Combining it with build ensures the application compiles completely from a clean slate, preventing stale or corrupted cache files from causing runtime errors.

---

## Step-by-Step Procedure

### Phase 1: Base Maven Project Preparation

- **Step 1:** Create a standard Maven Project inside Eclipse following the exact steps outlined in **Program 2**.
- **Step 2:** Compile and run the Java application to verify it outputs the standard *"Hello World!"* message in the console.

### Phase 3: Automatic Migration via Terminal

- **Step 3:** Open the terminal window within Eclipse using the shortcut key combination:

**Ctrl + Alt + Shift + T** (Alternatively, open the Windows Command Prompt and navigate to the project directory).

- **Step 4:** Initiate the automatic conversion by running the initialization command:

DOS

gradle init

- **Step 5:** When the prompt asks whether to migrate the project from Maven to Gradle, type **yes** and press **Enter**.
- **Step 6:** When prompted to select the Domain Specific Language (DSL), select **Option 2** to generate a **Kotlin DSL** build script (build.gradle.kts).
- **Step 7:** When the prompt asks to validate and generate the API project configuration, type **yes** and press **Enter** to finalize the init phase.

### Phase 3: Configuring the Simplified Build Script

- **Step 8:** Locate and open the newly generated **build.gradle.kts** file from your project directory.
- **Step 9:** Clear out the long generated file and replace its contents with this clean, exam-safe minimal block:

Kotlin

plugins {

    id("application")                      // Activates the application execution tasks

}

application {

    mainClass.set("com.pgm4.test.App") // Links to your exact Java package and main class

}

repositories {

```
mavenCentral()           // Configures the remote repository for
dependencies
}
```

#### Phase 4: Compiling and Execution

- **Step 10:** Return to your command prompt/terminal execution path and execute the clean-up and build task sequence:

DOS

gradle clean build

- **Step 11:** Once the build successfully compiles without errors, launch the migrated application using the Gradle execute target:

DOS

gradle run

- **Step 12:** Verify the successfully migrated console output sequence:

Plaintext

Hello World! Welcome to pgm4

### PROGRAM-5

**Title:** Introduction to Jenkins: What is Jenkins?, Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use.

#### Viva / Theory Questions

**Q1. What is Jenkins and what is its role in DevOps?** **Ans:** Jenkins is an open-source, Java-based automation server used to implement Continuous Integration (CI) and Continuous Delivery (CD) pipelines. It automates the software development workflow, including building, testing, and deploying code changes into production.

**Q2. How do you locate the initial administrator password to unlock Jenkins after installation?** **Ans:** During the first startup, Jenkins generates a unique random password for security. On Windows, this password is saved in a specific hidden system file path:

C:\ProgramData\Jenkins\jenkins\secrets\initialAdminPassword

The administrator must open this file using a text editor (like Notepad), copy the key, and paste it into the Jenkins setup wizard.

---

#### Step-by-Step Procedure

##### Phase 1: Downloading and Installing Jenkins

- **Step 1:** Open a browser and visit the official Jenkins download page: <https://www.jenkins.io/download/>. Download the **Windows (MSI)** installer package.
- **Step 2:** Locate and double-click the downloaded Jenkins MSI file to launch the setup installation wizard. Click **Next**.
- **Step 3:** In the Service Logon Credentials screen, select the option "**Run service as Local System**". Leave the Account and Password fields blank and click **Next**.
- **Step 4:** Set the default port to **8080**. Click the **Test Port** button to ensure the port is free (indicated by a green checkmark), then click **Next**.
- **Step 5 (JDK Verification):** The installer will scan for your Java installation. Open a new Command Prompt and verify your current version by typing:

DOS

java -version

*(Note: Jenkins requires JDK 17 to 21. If it fails to detect a compatible version, download and install JDK 21 from Oracle's archive archive page).*

- **Step 6:** Browse and select the valid JDK path in the wizard setup, then click **Next** -> **Next**.
- **Step 7:** Accept the Java Automation Server prompt, click **Install**, and once the setup completes, click **Finish**.

## Phase 2: Unlocking Jenkins and Initial Configuration

- **Step 8:** Open your web browser and navigate to the Jenkins local server address:

http://localhost:8080/

- **Step 9:** The "**Unlock Jenkins**" screen will appear requesting an Administrator password.
- **Step 10:** Open Windows File Explorer and navigate to the secret directory path:

C:\ProgramData\Jenkins\.jenkins\secrets\

*(Tip: ProgramData is a hidden folder, type the path directly into the File Explorer address bar if it is not visible).*

- **Step 11:** Open the file named **initialAdminPassword** with Notepad, copy the alphanumeric key string, paste it into the Jenkins browser setup page, and click **Continue**.

## Phase 3: Plugin Setup and Completing the Wizard

- **Step 12:** In the setup wizard customizing screen, click on the "**Install Suggested Plugins**" button.
- **Step 13:** Wait for the server to automatically download and configure core components (Git, Pipeline, Gradle, etc.).
- **Step 14:** When prompted to create the First Admin User, click "**Skip and continue as admin**" (or fill out the registry fields).
- **Step 15:** Save the default Jenkins URL path and click "**Start using Jenkins**" to reach the main dashboard screen.

## PROGRAM-7

**Title:** Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook.

### Viva / Theory Questions

**Q1. What is the difference between an Ansible Control Node and a Managed Host?** **Ans:** The **Control Node** is the system where Ansible is installed, configurations are written, and execution commands originate. The **Managed Hosts** are the remote servers or local environments that are controlled, configured, and orchestrated by the control node. Ansible does not require an agent to be running on the managed hosts; it communicates over standard SSH.

**Q2. What is an Ansible Inventory file and where is it located by default?** **Ans:** The Inventory file (typically located at /etc/ansible/hosts) lists the IP addresses or domain names of the target servers managed by Ansible. It allows administrators to organize individual nodes into categorized logical groups (e.g., [webservers], [databases]) for selective execution.

---

### Step-by-Step Procedure

## Phase 1: Configuring the Ansible Control Node User

- **Step 1:** Open the Ubuntu terminal on your main server (Control Node). Create a dedicated administrative user account for Ansible management:

Bash

```
sudo adduser ansible
```

- **Step 2:** Provide a strong password when prompted, and press **Enter** to skip optional demographic details.
- **Step 3:** Assign superuser privileges to the newly created user by appending it to the sudo group:

Bash

```
sudo usermod -aG sudo ansible
```

- **Step 4:** Switch active system context to the newly created Ansible user workspace:

Bash

```
sudo su ansible
```

## Phase 2: Generating and Distributing SSH Keys

- **Step 5:** Generate an authentication SSH cryptographic key pair on the control node:

Bash

```
ssh-keygen
```

*(Press Enter to save to the default directory and optionally press Enter twice to bypass passphrase assignment).*

- **Step 6:** Identify the IP address of your remote managed host machine by executing this command on the host terminal:

Bash

```
hostname -i
```

- **Step 7:** From the Control Node, securely transfer the public authentication key to the target host system account (e.g., remote IP 192.168.0.81):

Bash

```
ssh-copy-id ansible@192.168.0.81
```

- **Step 8:** Type **yes** to approve connection authenticity, enter the host password, and wait for confirmation that the key has been appended successfully.

## Phase 3: Installing and Verifying Ansible

- **Step 9:** Update the host native repository indexing tables via the APT packaging infrastructure:

Bash

```
sudo apt update
```

- **Step 10:** Execute the localized environment installation script deployment command:

Bash

```
sudo apt install ansible -y
```

- **Step 11:** Verify successful system installation and check system path details:

Bash

```
ansible --version
```

## Phase 4: Setting up the Inventory Configurations



- **Step 12:** Create the centralized configuration asset folder system directory layout structure:

Bash

```
sudo mkdir -p /etc/ansible
```

- **Step 13:** Launch the Nano text editor engine to create or alter the primary baseline static hosts database entry sheet:

Bash

```
sudo nano /etc/ansible/hosts
```

- **Step 14:** Append the local loopback execution parameters into a designated configuration container group structure:

Ini, TOML

```
[local]
```

```
localhost ansible_connection=local
```

*(Note: You can also add your remote server group below it, such as [servers] followed by the host IP address 192.168.0.81)*

- **Step 15:** Press **Ctrl + O** then **Enter** to save changes, and **Ctrl + X** to exit the Nano editor.
- **Step 16:** Review active parsed configuration inventory entries using structural verification layouts:

Bash

```
ansible-inventory --list -y
```

### Phase 5: Executing Connectivity Checks (Ping Test)

- **Step 17:** Invoke a global execution trace testing verification ping pattern request to confirm setup integrity across nodes:

Bash

```
sudo ansible all -m ping
```

- **Step 18:** Confirm that the output text displays a green or yellow structure showing "ping": "pong" and "changed": false, which indicates a successful setup.

6) Here is the exact, complete sequence of actions to execute right now on your screen. Follow this list from top to bottom:

### Phase 1: Install the Maven Integration Plugin

1. Open your browser to <http://localhost:8080>.
2. Click **Manage Jenkins** on the left menu.
3. Click on **Plugins**.
4. Click the **Available plugins** tab on the left.
5. In the search bar at the top, type: maven integration plugin.
6. Check the box next to it and click the **Install** button on the top right. Wait for it to finish downloading.

### Phase 2: Link Your Local Maven Software Path

1. Click **Manage Jenkins** on the left menu again.
2. Click on **Tools**.
3. Scroll down to the **Maven** section and click **Maven installations....**
4. Click **Add Maven**.
5. Change the **Name** field to exactly: MAVEN\_HOME.

6. **Uncheck** the box that says "Install automatically".
7. In the **MAVEN\_HOME** input field, paste the exact folder path where your Maven package sits on your laptop:

Plaintext

C:\Program Files\apache-maven-3.9.16

8. Click the blue **Save** button at the bottom.

### Phase 3: Create and Configure Your Freestyle Job

1. Click **New Item** at the top-left of the Jenkins home page.
2. In the text field, type the item name exactly: Maven Test.
3. Select the option **Freestyle project** (do not select Maven Project).
4. Click the blue **OK** button at the bottom.
5. On the settings page that loads, scroll down to the **Build Steps** section.
6. Click **Add build step** and select **Invoke top-level Maven targets**.
7. Change the **Maven Version** dropdown selection from *(Default)* to **MAVEN\_HOME**.
8. In the **Goals** text box, type exactly:

Plaintext

clean install

9. Click the **Advanced...** button right below the goals box.
10. In the **POM** text input box, paste your exact absolute Windows project file path:

Plaintext

C:\Users\chakr\eclipse-workspace\program2-example-jar\pom.xml

11. Scroll to the very bottom of the page and click the blue **Save** button.

### Phase 4: Run and Verify the Output

1. On your new project page, click **Build Now** on the left-hand menu.
2. Look at the lower-left section under **Build History** and click on your newest build number (#1).
3. Click **Console Output** on the left sidebar.

Scroll to the very bottom of the log window. You will see the clean Maven compilation matrix tracking your classes and a final green BUILD SUCCESSFUL status! Turn this on now and watch it pass.

Here is your ultimate, quick-reference Viva guide for Programs 1 to 7. Each experiment is broken down into exactly **what it is** (the core concept) and **what you did** in the lab, phrased perfectly so you can confidently answer any external examiner.

## 1. Introduction to Maven and Gradle

- **What it is:** This program introduces **Build Automation Tools**. These tools automatically compile source code, download required external libraries (dependencies), run tests, and package code into executable files (like .jar or .war).

- **Key Viva Difference:** **Maven** uses XML (pom.xml) and follows a strict, rigid lifecycle. **Gradle** uses code-based scripts (Groovy/Kotlin) and is much faster because it only compiles files that have changed (incremental builds).
- **What we did:** We explored how these tools are structured and installed their core software binaries on the system.

## 2. Working with Maven

- **What it is:** A deep dive into Maven's project ecosystem. The heart of any Maven project is the **POM (Project Object Model)** file.
- **What we did:** We created a Java project inside Eclipse using Maven. We used the pom.xml file to declare dependencies (like JUnit for testing). When we ran the project, Maven automatically went online, fetched those libraries, and managed them for us.

## 3. Working with Gradle

- **What it is:** A deep dive into Gradle's build engine. Instead of rigid XML, it uses flexible build scripts written in **Groovy DSL**. It handles automation via **Tasks** (individual units of work, like compiling or cleaning).
- **What we did:** We initialized a Gradle workspace, examined its build scripts, and saw how it automates execution sequences through specialized commands in the terminal pane.

## 4. Practical Exercise: Build Migration (Maven to Gradle)

- **What it is:** Learning how to migrate an existing production project from one build ecosystem to another without breaking the application logic.
- **What we did:** We took our working Maven Java application from Program 2, opened the terminal inside Eclipse, and ran gradle init. We converted the pom.xml structure into a fresh build.gradle script, customized the main class path target, and executed it using gradle clean build and gradle run to output our final console message.

## 5. Introduction to Jenkins

- **What it is:** Jenkins is an open-source **Automation Server** used to implement Continuous Integration and Continuous Deployment (CI/CD). It monitors code repositories and automatically triggers workflows whenever changes occur.
- **What we did:** We downloaded the Jenkins .msi installer for Windows, configured it to run as a Local System service on port 8080, unlocked it using the secret administrator password hidden in the system folders, and installed the default base plugins.

## 6. Continuous Integration with Jenkins

- **What it is:** **Continuous Integration (CI)** is the DevOps practice of automatically compiling, building, and testing a software project every single time a developer modifies the code.
- **What we did:** We installed the **Maven Integration Plugin** inside Jenkins. We created a **Freestyle Project** job, pointed it directly to our local project's pom.xml absolute Windows file path, set the goal target to clean install, and ran **Build Now** to watch Jenkins automatically compile our Java project and test it hands-free.

## 7. Configuration Management with Ansible

- **What it is:** Ansible is an IT automation tool used for **Configuration Management**. It allows you to automatically deploy apps, install software packages, and configure settings across hundreds of remote servers simultaneously using simple text scripts. It is **agentless** (connects via standard SSH).

- **What we did:** We opened our Ubuntu/WSL Linux terminal on Windows and installed Ansible along with OpenSSH. We created an **Inventory** file (hosts) specifying our target computer (localhost). Then, we wrote an automated script called a **Playbook** (playbook.yml) that automatically printed a console message and instantly generated a text file on the drive using ansible-playbook.

### 💡 Quick Viva Tip for DevOps:

If the examiner asks why we use all these tools together:

*"We use **Maven/Gradle** to build individual applications, **Jenkins** to automatically trigger those builds whenever code changes (Continuous Integration), and **Ansible** to automatically set up and configure the target deployment servers (Configuration Management)."*

Here is exactly how to explain a **Pipeline** to your examiner, along with a complete master list of all the DevOps abbreviations and full forms you need to know for your viva.

### 1. What is a Pipeline? (To Answer in Viva)

If the examiner asks, "**What is a pipeline in DevOps?**", give them this precise definition:

"A **Pipeline** is a series of automated steps that software code goes through from development to production. It connects different stages of software delivery—such as pulling code, compiling, running unit tests, packaging, and deploying—so that code can be delivered automatically, quickly, and without human error."

#### The Key Stages of a CI/CD Pipeline:

- **Source Stage:** Triggered automatically when code is modified or pushed (e.g., via GitHub).
- **Build Stage:** Compiles the source files and resolves external packages (e.g., via Maven or Gradle).
- **Test Stage:** Automatically runs unit tests to catch code bugs before deployment (e.g., via JUnit).
- **Deploy Stage:** Automatically delivers the packaged code to production environments or staging servers.

### 2. DevOps Master Full-Form Checklist

Here are all the crucial abbreviations you encountered across your 7 lab experiments:

#### Core DevOps & Tool Terms

- **CI:** Continuous Integration (*Automating the building and testing of code updates*)
- **CD:** Continuous Delivery or Continuous Deployment (*Automating the release of code to servers*)
- **PPA:** Personal Package Archive (*Used in Ubuntu to add custom software repositories, like we did for Ansible*)
- **SSH:** Secure Shell (*The encrypted network protocol Ansible uses to connect and manage target servers*)
- **YAML:** YAML Ain't Markup Language (*The human-readable data format used to write Ansible Playbooks*)

- **DSL:** Domain Specific Language (*The language structure used to write Gradle build scripts, like Groovy or Kotlin*)

### **Java & Build Management Terms**

- **POM:** Project Object Model (*The central XML configuration file used in Maven projects*)
- **XML:** Extensible Markup Language (*The tag-based language format used to write Maven pom.xml files*)
- **JDK:** Java Development Kit (*The underlying software environment package used to compile Java programs*)
- **JAR:** Java Archive (*The compressed executable package file format that packs Java classes together*)
- **WAR:** Web Archive (*The package format used to distribute web applications across servers*)
- **URL:** Uniform Resource Locator (*The web address path used to navigate to repositories or web interfaces like Jenkins*)



### **Extra Viva Cheat Sheet:**

If the examiner asks, "**What tool acts as the pipeline manager in your lab?**" >

**Answer:** "*Jenkins acts as our central pipeline orchestration engine. It automates the whole lifecycle by calling **Maven/Gradle** to build the files, and it can hand over control to **Ansible** to manage the server deployment.*"